

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```
In [2]: customer =
pd.read_csv("E:\Quantities_Data_Analyst_Project\QVI_purchase_behaviour.csv")
transaction =
pd.read_csv("E:\Quantities_Data_Analyst_Project\QVI_transaction_data.csv")
```

```
In [3]: customer.head()
```

```
Out[3]:
```

	LYLTY_CARD_NBR	LIFESTAGE	PREMIUM_CUSTOMER
0	1000	YOUNG SINGLES/COUPLES	Premium
1	1002	YOUNG SINGLES/COUPLES	Mainstream
2	1003	YOUNG FAMILIES	Budget
3	1004	OLDER SINGLES/COUPLES	Mainstream
4	1005	MIDAGE SINGLES/COUPLES	Mainstream

```
In [4]: customer.tail()
```

```
Out[4]:
```

	LYLTY_CARD_NBR	LIFESTAGE	PREMIUM_CUSTOMER
72632	2370651	MIDAGE SINGLES/COUPLES	Mainstream
72633	2370701	YOUNG FAMILIES	Mainstream
72634	2370751	YOUNG FAMILIES	Premium
72635	2370961	OLDER FAMILIES	Budget
72636	2373711	YOUNG SINGLES/COUPLES	Mainstream

```
In [5]: transaction.head()
```

Out[5]:

	DATE	STORE_NBR	LYLTY_CARD_NBR	TXN_ID	PROD_NBR	PROD_NAME	PROD_QTY	TOT_S/
0	17-10-2018	1	1000	1	5	Natural Chip Compny SeaSalt175g	2	
1	14-05-2019	1	1307	348	66	CCs Nacho Cheese 175g	3	
2	20-05-2019	1	1343	383	61	Smiths Crinkle Cut Chips Chicken 170g	2	
3	17-08-2018	2	2373	974	69	Smiths Chip Thinly S/Cream&Onion 175g	5	
4	18-08-2018	2	2426	1038	108	Kettle Tortilla ChpsHny&Jlpno Chili 150g	3	

In [6]: `transaction.tail()`

Out[6]:

	DATE	STORE_NBR	LYLTY_CARD_NBR	TXN_ID	PROD_NBR	PROD_NAME	PROD_QTY	TOT_S/
264831	09-03-2019	272	272319	270088	89	Kettle Sweet Chilli And Sour Cream 175g	2	
264832	13-08-2018	272	272358	270154	74	Tostitos Splash Of Lime 175g	1	
264833	06-11-2018	272	272379	270187	51	Doritos Mexicana 170g	2	
264834	27-12-2018	272	272379	270188	42	Doritos Corn Chip Mexican Jalapeno 150g	2	
264835	22-09-2018	272	272380	270189	74	Tostitos Splash Of Lime 175g	2	

In [7]: `customer.columns`

Out[7]: `Index(['LYLTY_CARD_NBR', 'LIFESTAGE', 'PREMIUM_CUSTOMER'], dtype='object')`

In [8]: `transaction.columns`

Out[8]: `Index(['DATE', 'STORE_NBR', 'LYLTY_CARD_NBR', 'TXN_ID', 'PROD_NBR', 'PROD_NAME', 'PROD_QTY', 'TOT_SALES'], dtype='object')`

In [9]: `customer.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 72637 entries, 0 to 72636
Data columns (total 3 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   LYLTY_CARD_NBR        72637 non-null  int64
1   LIFESTAGE              72637 non-null  object
2   PREMIUM_CUSTOMER      72637 non-null  object
dtypes: int64(1), object(2)
memory usage: 1.7+ MB
```

In [10]: `transaction.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 264836 entries, 0 to 264835
Data columns (total 8 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   DATE                  264836 non-null  object
1   STORE_NBR             264836 non-null  int64
2   LYLTY_CARD_NBR        264836 non-null  int64
3   TXN_ID                264836 non-null  int64
4   PROD_NBR              264836 non-null  int64
5   PROD_NAME             264836 non-null  object
6   PROD_QTY              264836 non-null  int64
7   TOT_SALES             264836 non-null  float64
dtypes: float64(1), int64(5), object(2)
memory usage: 16.2+ MB
```

In [11]: `customer.describe()`

Out[11]:

	LYLTY_CARD_NBR
count	7.263700e+04
mean	1.361859e+05
std	8.989293e+04
min	1.000000e+03
25%	6.620200e+04
50%	1.340400e+05
75%	2.033750e+05
max	2.373711e+06

In [12]: `transaction.describe()`

Out[12]:	STORE_NBR	LYLTY_CARD_NBR	TXN_ID	PROD_NBR	PROD_QTY	TOT_SALES
count	264836.00000	2.648360e+05	2.648360e+05	264836.000000	264836.000000	264836.000000
mean	135.08011	1.355495e+05	1.351583e+05	56.583157	1.907309	7.304200
std	76.78418	8.057998e+04	7.813303e+04	32.826638	0.643654	3.083226
min	1.00000	1.000000e+03	1.000000e+00	1.000000	1.000000	1.500000
25%	70.00000	7.002100e+04	6.760150e+04	28.000000	2.000000	5.400000
50%	130.00000	1.303575e+05	1.351375e+05	56.000000	2.000000	7.400000
75%	203.00000	2.030942e+05	2.027012e+05	85.000000	2.000000	9.200000
max	272.00000	2.373711e+06	2.415841e+06	114.000000	200.000000	650.000000

In [13]: *#checking Null values*

In [14]: `customer.isnull().sum()`

Out[14]:

LYLTY_CARD_NBR	0
LIFESTAGE	0
PREMIUM_CUSTOMER	0
dtype: int64	

In [15]: `transaction.isnull().sum()`

Out[15]:

DATE	0
STORE_NBR	0
LYLTY_CARD_NBR	0
TXN_ID	0
PROD_NBR	0
PROD_NAME	0
PROD_QTY	0
TOT_SALES	0
dtype: int64	

In [16]: *# checking Duplicated Values*

In [17]: `customer.duplicated().sum()`

Out[17]: 0

In [18]: `transaction.duplicated().sum()`

Out[18]: 1

In [19]: `transaction.drop_duplicates(inplace= True)`

In [20]: `transaction.duplicated().sum()`

Out[20]: 0

```
In [21]: customer.nunique()
```

```
Out[21]: LYLTY_CARD_NBR      72637  
LIFESTAGE              7  
PREMIUM_CUSTOMER      3  
dtype: int64
```

```
In [22]: transaction.nunique()
```

```
Out[22]: DATE              364  
STORE_NBR              272  
LYLTY_CARD_NBR      72637  
TXN_ID              263127  
PROD_NBR              114  
PROD_NAME            114  
PROD_QTY              6  
TOT_SALES            112  
dtype: int64
```

```
In [23]: customer['LIFESTAGE'].unique()
```

```
Out[23]: array(['YOUNG SINGLES/COUPLES', 'YOUNG FAMILIES', 'OLDER SINGLES/COUPLES',  
                'MIDAGE SINGLES/COUPLES', 'NEW FAMILIES', 'OLDER FAMILIES',  
                'RETIREEES'], dtype=object)
```

```
In [24]: customer['PREMIUM_CUSTOMER'].unique()
```

```
Out[24]: array(['Premium', 'Mainstream', 'Budget'], dtype=object)
```

```
In [25]: customer['LYLTY_CARD_NBR'].duplicated().sum()
```

```
Out[25]: 0
```

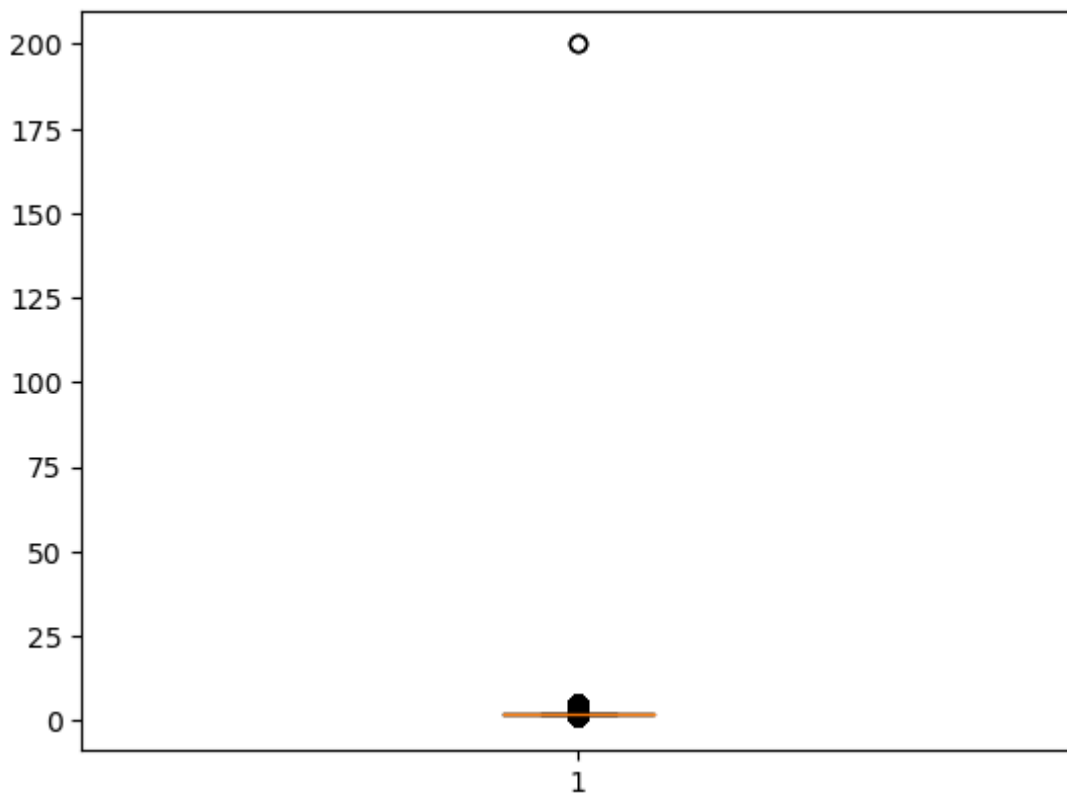
```
In [26]: # Checking Date Range and Gap in Date Columns
```

```
In [27]: Min_Date = transaction['DATE'].min()  
Max_Date = transaction['DATE'].max()  
full_range = pd.date_range(Max_Date, Min_Date)  
missing_date = full_range.difference(transaction['DATE'].unique())  
print(missing_date)
```

```
DatetimeIndex([], dtype='datetime64[ns]', freq=None)
```

```
In [28]: # Checking outlier
```

```
In [29]: plt.boxplot(transaction['PROD_QTY'])  
plt.show()
```



```
In [30]: transaction['PROD_QTY'].unique()
```

```
Out[30]: array([ 2,  3,  5,  1,  4, 200], dtype=int64)
```

```
In [31]: # Filltering all records with 200 prod qty
```

```
In [32]: transaction[transaction['PROD_QTY'] > 10].value_counts()
```

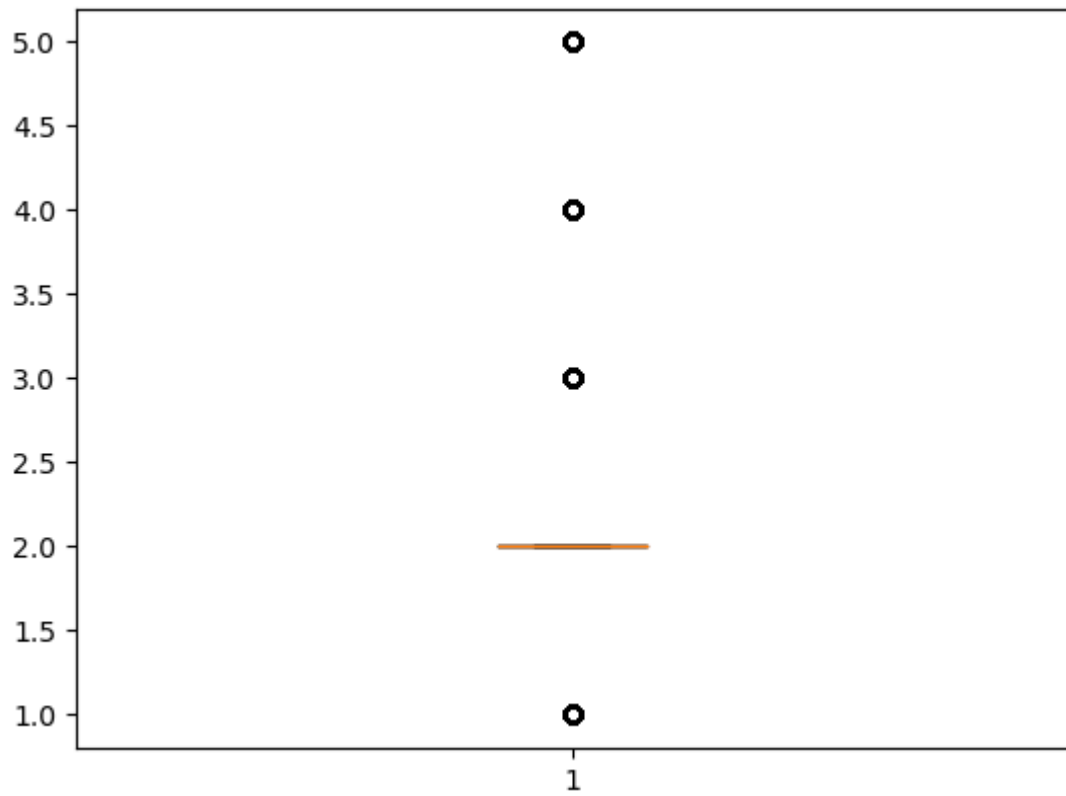
```
Out[32]:
```

DATE	STORE_NBR	LYLTY_CARD_NBR	TXN_ID	PROD_NBR	PROD_NAME
19-08-2018	226	226000	226201	4	Dorito Corn Chp
me 380g	200	650.0	1		Supre
20-05-2019	226	226000	226210	4	Dorito Corn Chp
me 380g	200	650.0	1		Supre

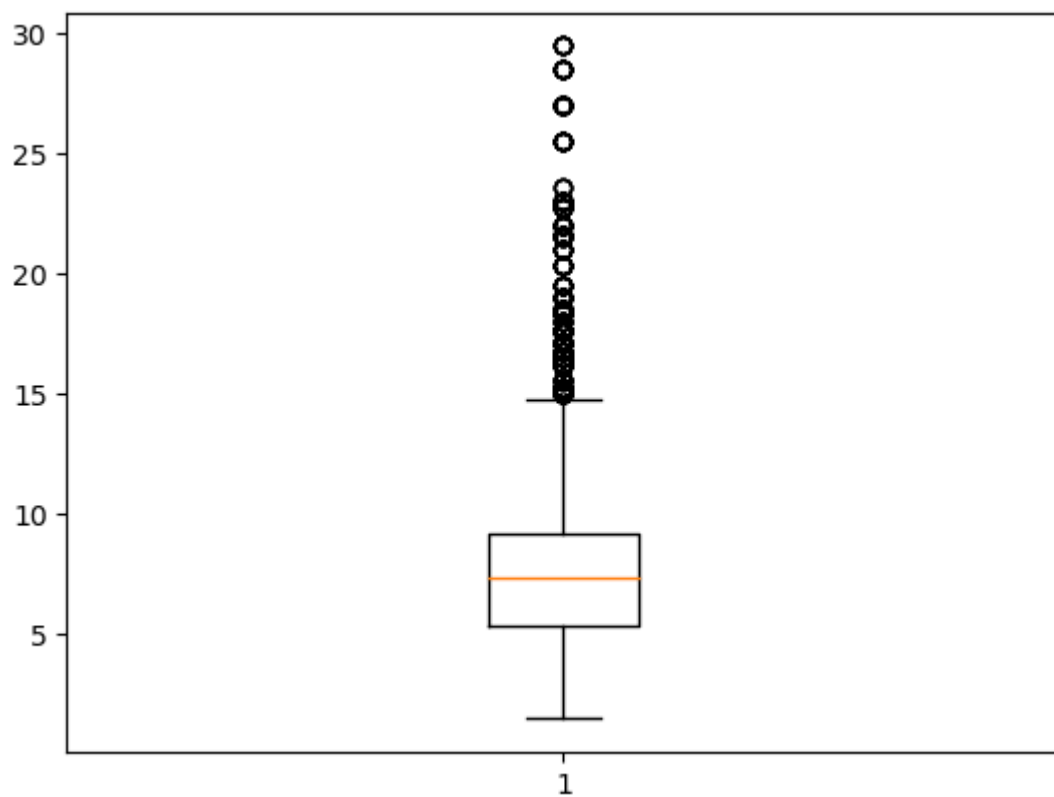
Name: count, dtype: int64

```
In [33]: transaction = transaction[transaction['PROD_QTY'] < 10]
```

```
In [34]: plt.boxplot(transaction['PROD_QTY'])
plt.show()
```



```
In [35]: plt.boxplot(transaction['TOT_SALES'])  
plt.show()
```



```
In [36]: transaction['PROD_NAME'].head(10)
```

```
Out[36]: 0    Natural Chip      Compny SeaSalt175g
1              CCs Nacho Cheese      175g
2    Smiths Crinkle Cut  Chips Chicken 170g
3    Smiths Chip Thinly  S/Cream&Onion 175g
4    Kettle Tortilla ChpsHny&Jlpno Chili 150g
5    Old El Paso Salsa   Dip Tomato Mild 300g
6    Smiths Crinkle Chips Salt & Vinegar 330g
7      Grain Waves      Sweet Chilli 210g
8    Doritos Corn Chip Mexican Jalapeno 150g
9      Grain Waves Sour   Cream&Chives 210G
Name: PROD_NAME, dtype: object
```

```
In [37]: # Feature Engineering
```

```
In [38]: transaction["PACK_SIZE"] =
transaction["PROD_NAME"].str.extract('(\d+)').astype(int)
```

```
In [39]: transaction['BRAND'] = transaction['PROD_NAME'].str.split().str[0]
```

```
In [40]: transaction.head()
```

```
Out[40]:
```

	DATE	STORE_NBR	LYLTY_CARD_NBR	TXN_ID	PROD_NBR	PROD_NAME	PROD_QTY	TOT_SA
0	17-10-2018	1	1000	1	5	Natural Chip Compny SeaSalt175g	2	
1	14-05-2019	1	1307	348	66	CCs Nacho Cheese 175g	3	
2	20-05-2019	1	1343	383	61	Smiths Crinkle Cut Chips Chicken 170g	2	
3	17-08-2018	2	2373	974	69	Smiths Chip Thinly S/Cream&Onion 175g	5	
4	18-08-2018	2	2426	1038	108	Kettle Tortilla ChpsHny&Jlpno Chili 150g	3	

```
In [41]: transaction["PACK_SIZE"]
```



```
Out[41]:
0      175
1      175
2      170
3      175
4      150
...
264831  175
264832  175
264833  170
264834  150
264835  175
Name: PACK_SIZE, Length: 264833, dtype: int32
```

```
In [42]: transaction["BRAND"].value_counts().sort_index()
```

```
Out[42]:
BRAND
Burger      1564
CCs         4551
Cheetos     2927
Cheezels    4603
Cobs        9693
Dorito       3183
Doritos     24962
French       1418
Grain        6272
GrnWves     1468
Infuzions   11057
Infzns      3144
Kettle      41288
NCC          1419
Natural     6050
Old          9324
Pringles    25102
RRD         11894
Red          5885
Smith        2963
Smiths      28859
Snbts       1576
Sunbites    1432
Thins       14075
Tostitos    9471
Twisties    9454
Tyrrells    6442
WW          10320
Woolworths  4437
Name: count, dtype: int64
```

```
In [43]: transaction[transaction["BRAND"] == "Old"]["PROD_NAME"].unique()
```

```
Out[43]:
array(['Old El Paso Salsa   Dip Tomato Mild 300g',
      'Old El Paso Salsa   Dip Chnky Tom Ht300g',
      'Old El Paso Salsa   Dip Tomato Med 300g'], dtype=object)
```

```
In [44]: transaction.loc[
        transaction["PROD_NAME"].str.startswith("Natural Chip Compny"),
```

```

    "BRAND"
] = "Natural Chip Compny"

transaction.loc[
    transaction["PROD_NAME"].str.startswith("Old El Paso"),
    "BRAND"
] = "Old El Paso"

transaction.loc[
    transaction["PROD_NAME"].str.startswith("Grain Waves"),
    "BRAND"
] = "Grain Waves"

transaction.loc[
    transaction["PROD_NAME"].str.startswith("Red Rock Deli"),
    "BRAND"
] = "Red Rock Deli"

```

In [45]: `transaction.head()`

Out[45]:

	DATE	STORE_NBR	LYLTY_CARD_NBR	TXN_ID	PROD_NBR	PROD_NAME	PROD_QTY	TOT_SA
0	17-10-2018	1	1000	1	5	Natural Chip Compny SeaSalt175g	2	
1	14-05-2019	1	1307	348	66	CCs Nacho Cheese 175g	3	
2	20-05-2019	1	1343	383	61	Smiths Crinkle Cut Chips Chicken 170g	2	
3	17-08-2018	2	2373	974	69	Smiths Chip Thinly S/Cream&Onion 175g	5	
4	18-08-2018	2	2426	1038	108	Kettle Tortilla ChpsHny&Jlpno Chili 150g	3	

In [46]: `# Merging both datasets`

In [47]: `df_final = transaction.merge(customer,on = "LYLTY_CARD_NBR", how = 'left')`

In [48]: `df_final.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 264833 entries, 0 to 264832
Data columns (total 12 columns):
#   Column                Non-Null Count  Dtype
---  -
0   DATE                   264833 non-null object
1   STORE_NBR              264833 non-null int64
2   LYLTY_CARD_NBR         264833 non-null int64
3   TXN_ID                 264833 non-null int64
4   PROD_NBR               264833 non-null int64
5   PROD_NAME              264833 non-null object
6   PROD_QTY               264833 non-null int64
7   TOT_SALES              264833 non-null float64
8   PACK_SIZE              264833 non-null int32
9   BRAND                  264833 non-null object
10  LIFESTAGE               264833 non-null object
11  PREMIUM_CUSTOMER       264833 non-null object
dtypes: float64(1), int32(1), int64(5), object(5)
memory usage: 23.2+ MB
```

```
In [49]: df_final.isnull().sum()
```

```
Out[49]: DATE                0
STORE_NBR                0
LYLTY_CARD_NBR          0
TXN_ID                   0
PROD_NBR                 0
PROD_NAME                0
PROD_QTY                 0
TOT_SALES                0
PACK_SIZE                0
BRAND                    0
LIFESTAGE                0
PREMIUM_CUSTOMER        0
dtype: int64
```

```
In [50]: df_final.duplicated().sum()
```

```
Out[50]: 0
```

```
In [51]: # Exploratory Data Analysis
```

```
In [52]: df_final.columns
```

```
Out[52]: Index(['DATE', 'STORE_NBR', 'LYLTY_CARD_NBR', 'TXN_ID', 'PROD_NBR',
               'PROD_NAME', 'PROD_QTY', 'TOT_SALES', 'PACK_SIZE', 'BRAND', 'LIFESTAGE',
               'PREMIUM_CUSTOMER'],
              dtype='object')
```

```
In [53]: # Total Sales
df_final['TOT_SALES'].sum()
```

```
Out[53]: 1933108.9999999998
```

In [54]:

Total Transactions

df_final['TXN_ID'].nunique()

Out[54]:

263125

In [55]:

total Customers

df_final['LYLTY_CARD_NBR'].nunique()

Out[55]:

72636

In [56]:

Total Qty Sold

df_final['PROD_QTY'].sum()

Out[56]:

504722

In [57]:

df_final.rename(columns=str.lower, inplace=True)

In [58]:

df_final.head()

Out[58]:

	date	store_nbr	lylty_card_nbr	txn_id	prod_nbr	prod_name	prod_qty	tot_sales	pack_size
0	17-10-2018	1	1000	1	5	Natural Chip Compny SeaSalt175g	2	6.0	17
1	14-05-2019	1	1307	348	66	CCs Nacho Cheese 175g	3	6.3	17
2	20-05-2019	1	1343	383	61	Smiths Crinkle Cut Chips Chicken 170g	2	2.9	17
3	17-08-2018	2	2373	974	69	Smiths Chip Thinly S/Cream&Onion 175g	5	15.0	17
4	18-08-2018	2	2426	1038	108	Kettle Tortilla ChpsHny&Jlpno Chili 150g	3	13.8	15

In [59]:

Created the segment column

```
df_final['segment'] = df_final['lifestage'] + " - " +
df_final['premium_customer']
```

In [60]:

df_final['segment'].head()

```
Out[60]: 0    YOUNG SINGLES/COUPLES - Premium
1    MIDGE SINGLES/COUPLES - Budget
2    MIDGE SINGLES/COUPLES - Budget
3    MIDGE SINGLES/COUPLES - Budget
4    MIDGE SINGLES/COUPLES - Budget
Name: segment, dtype: object
```

```
In [61]: # Total sales by segment

sales_by_segment = df_final.groupby(df_final['segment'])
['tot_sales'].sum().reset_index()
sales_by_segment = sales_by_segment.sort_values('tot_sales',ascending =
false)
sales_by_segment
```

```
Out[61]:
```

	segment	tot_sales
6	OLDER FAMILIES - Budget	168363.25
19	YOUNG SINGLES/COUPLES - Mainstream	157621.60
13	RETIREEES - Mainstream	155677.05
15	YOUNG FAMILIES - Budget	139345.85
9	OLDER SINGLES/COUPLES - Budget	136769.80
10	OLDER SINGLES/COUPLES - Mainstream	133393.80
11	OLDER SINGLES/COUPLES - Premium	132257.15
12	RETIREEES - Budget	113147.80
7	OLDER FAMILIES - Mainstream	103445.55
14	RETIREEES - Premium	97646.05
16	YOUNG FAMILIES - Mainstream	92788.75
1	MIDGE SINGLES/COUPLES - Mainstream	90803.85
17	YOUNG FAMILIES - Premium	84025.50
8	OLDER FAMILIES - Premium	80658.40
18	YOUNG SINGLES/COUPLES - Budget	61141.60
2	MIDGE SINGLES/COUPLES - Premium	58432.65
20	YOUNG SINGLES/COUPLES - Premium	41642.10
0	MIDGE SINGLES/COUPLES - Budget	35514.80
3	NEW FAMILIES - Budget	21928.45
4	NEW FAMILIES - Mainstream	17013.90
5	NEW FAMILIES - Premium	11491.10

```
In [69]: # Number of unique customers by segment

customers_by_segment = df_final.groupby(['segment'])
```

```
['lylty_card_nbr'].nunique().reset_index()  
customers_by_segment.columns = ['segment', 'num_cust']
```

In [70]: `customers_by_segment`

Out[70]:

	segment	num_cust
0	MIDAGE SINGLES/COUPLES - Budget	1504
1	MIDAGE SINGLES/COUPLES - Mainstream	3340
2	MIDAGE SINGLES/COUPLES - Premium	2431
3	NEW FAMILIES - Budget	1112
4	NEW FAMILIES - Mainstream	849
5	NEW FAMILIES - Premium	588
6	OLDER FAMILIES - Budget	4675
7	OLDER FAMILIES - Mainstream	2831
8	OLDER FAMILIES - Premium	2273
9	OLDER SINGLES/COUPLES - Budget	4929
10	OLDER SINGLES/COUPLES - Mainstream	4930
11	OLDER SINGLES/COUPLES - Premium	4750
12	RETIREEES - Budget	4454
13	RETIREEES - Mainstream	6479
14	RETIREEES - Premium	3872
15	YOUNG FAMILIES - Budget	4017
16	YOUNG FAMILIES - Mainstream	2728
17	YOUNG FAMILIES - Premium	2433
18	YOUNG SINGLES/COUPLES - Budget	3779
19	YOUNG SINGLES/COUPLES - Mainstream	8088
20	YOUNG SINGLES/COUPLES - Premium	2574

In [72]: *# Average spend per customer*

```
segment_summary = sales_by_segment.merge(customers_by_segment, on =  
'segment')  
segment_summary['Avg_spend_per_cust'] = segment_summary['tot_sales']/  
segment_summary['num_cust']
```

In [75]: `segment_summary.head()`

Out[75]:

	segment	tot_sales	num_cust	Avg_spend_per_cust
0	OLDER FAMILIES - Budget	168363.25	4675	36.013529
1	YOUNG SINGLES/COUPLES - Mainstream	157621.60	8088	19.488328
2	RETIREEES - Mainstream	155677.05	6479	24.027944
3	YOUNG FAMILIES - Budget	139345.85	4017	34.689034
4	OLDER SINGLES/COUPLES - Budget	136769.80	4929	27.747981

In [88]:

```
# Average transactions per customer (frequency)

taxn_by_segment = df_final.groupby('segment')
['txn_id'].nunique().reset_index()
taxn_by_segment.columns = ['segment', 'txn_num']
```

In [89]:

```
segment_summary = segment_summary.merge(taxn_by_segment, on = 'segment')
segment_summary['Avg_trans_per_cust'] =
segment_summary['txn_num']/segment_summary['num_cust']
```

In [86]:

```
segment_summary.head()
```

Out[86]:

	segment	tot_sales	num_cust	Avg_spend_per_cust	txn_id_x	txn_id_y	txn_num	Avg_
0	OLDER FAMILIES - Budget	168363.25	4675	36.013529	22935	22935	22935	
1	YOUNG SINGLES/COUPLES - Mainstream	157621.60	8088	19.488328	20783	20783	20783	
2	RETIREEES - Mainstream	155677.05	6479	24.027944	21363	21363	21363	
3	YOUNG FAMILIES - Budget	139345.85	4017	34.689034	18970	18970	18970	
4	OLDER SINGLES/COUPLES - Budget	136769.80	4929	27.747981	18301	18301	18301	

In [96]:

```
# Average units per transaction (basket size)

segment_summary['Avg_units_per_trans'] = df_final.groupby(['segment'])
['prod_qty'].sum().values/segment_summary['txn_num'].values
```

In [98]:

```
segment_summary.head()
```

Out[98]:

	segment	tot_sales	num_cust	Avg_spend_per_cust	txn_id_x	txn_id_y	txn_num_x	Avg
0	OLDER FAMILIES - Budget	168363.25	4675	36.013529	22935	22935	22935	
1	YOUNG SINGLES/COUPLES - Mainstream	157621.60	8088	19.488328	20783	20783	20783	
2	RETIREEES - Mainstream	155677.05	6479	24.027944	21363	21363	21363	
3	YOUNG FAMILIES - Budget	139345.85	4017	34.689034	18970	18970	18970	
4	OLDER SINGLES/COUPLES - Budget	136769.80	4929	27.747981	18301	18301	18301	

In [104...]

```
# Average price per unit
total_price = df_final.groupby('segment')['tot_sales'].sum().values
total_qty = df_final.groupby('segment')['prod_qty'].sum().values
segment_summary['Avg_price_per_unit'] = total_price/total_qty
```

In [105...]

```
segment_summary.head()
```

Out[105]:

	segment	tot_sales	num_cust	Avg_spend_per_cust	txn_id_x	txn_id_y	txn_num_x	Avg
0	OLDER FAMILIES - Budget	168363.25	4675	36.013529	22935	22935	22935	
1	YOUNG SINGLES/COUPLES - Mainstream	157621.60	8088	19.488328	20783	20783	20783	
2	RETIREEES - Mainstream	155677.05	6479	24.027944	21363	21363	21363	
3	YOUNG FAMILIES - Budget	139345.85	4017	34.689034	18970	18970	18970	
4	OLDER SINGLES/COUPLES - Budget	136769.80	4929	27.747981	18301	18301	18301	

In [107...]

```
df_final.columns
```

Out[107]:

```
Index(['date', 'store_nbr', 'lylty_card_nbr', 'txn_id', 'prod_nbr',
       'prod_name', 'prod_qty', 'tot_sales', 'pack_size', 'brand', 'lifestage',
       'premium_customer', 'segment'],
      dtype='object')
```

In [106...]

```
# Preferred pack size by segment
```

In [118...]

```
pack_seg = df_final.groupby(['segment', 'pack_size'])
['prod_qty'].sum().reset_index()
```



```

# total units sold of each pack size across ALL customers (baseline)

pack_total = df_final.groupby(['pack_size'])
['prod_qty'].sum().reset_index()
pack_total.columns = ['pack_size', 'total_prod_qty']
total_qty = df_final['prod_qty'].sum()
pack_total['pack_share_overall'] =
pack_total['total_prod_qty']/total_qty
#pack_total.head()

seg_total_unit = df_final.groupby('segment')
['prod_qty'].sum().reset_index()
seg_total_unit.columns = ['segment', 'total_unit_segment']
#seg_total_unit

pack_seg = pack_seg.merge(seg_total_unit, on =
'segment').merge(pack_total, on='pack_size')
# pack_seg.head()

pack_seg['pack_share_segment'] =
pack_seg['prod_qty']/pack_seg['total_unit_segment']
pack_seg['affinity_index'] = pack_seg['pack_share_segment'] /
pack_seg['pack_share_overall'] * 100
pack_seg.head()

```

Out[118]:

	segment	pack_size	prod_qty	total_unit_segment	total_prod_qty	pack_share_overall	
0	MIDAGE SINGLES/COUPLES - Budget	70	51	9496	2855	0.005657	
1	MIDAGE SINGLES/COUPLES - Mainstream	70	96	22699	2855	0.005657	
2	MIDAGE SINGLES/COUPLES - Premium	70	117	15526	2855	0.005657	
3	NEW FAMILIES - Budget	70	25	5571	2855	0.005657	
4	NEW FAMILIES - Mainstream	70	15	4319	2855	0.005657	

In [128...

```

# Preferred brand by segment

brand_seg = df_final.groupby(['segment', 'brand'])

```

```

['prod_qty'].sum().reset_index()

# total units sold of each brand across ALL customers (baseline)

brand_total = df_final.groupby('brand')['prod_qty'].sum().reset_index()
brand_total.columns = ['brand', 'total_qty_sold']
#brand_total.head()

total_qty = df_final['prod_qty'].sum()

brand_total['brand_share_overall'] =
brand_total['total_qty_sold']/total_qty

#brand_total.head()
#seg_total_unit.head()

brand_seg =
brand_seg.merge(seg_total_unit, on='segment').merge(brand_total, on='brand')
#brand_seg.head()

brand_seg['brand_share_segment']
=brand_seg['prod_qty']/brand_seg['total_unit_segment']
brand_seg['affinity_index'] =
brand_seg['brand_share_segment']/brand_seg['brand_share_overall'] * 100

brand_seg.head()

```

Out[128]:

	segment	brand	prod_qty	total_unit_segment	total_qty_sold	brand_share_overall	bra
0	MIDAGE SINGLES/COUPLES - Budget	Burger	84	9496	2970	0.005884	
1	MIDAGE SINGLES/COUPLES - Mainstream	Burger	92	22699	2970	0.005884	
2	MIDAGE SINGLES/COUPLES - Premium	Burger	111	15526	2970	0.005884	
3	NEW FAMILIES - Budget	Burger	30	5571	2970	0.005884	
4	NEW FAMILIES - Mainstream	Burger	27	4319	2970	0.005884	

In [133...

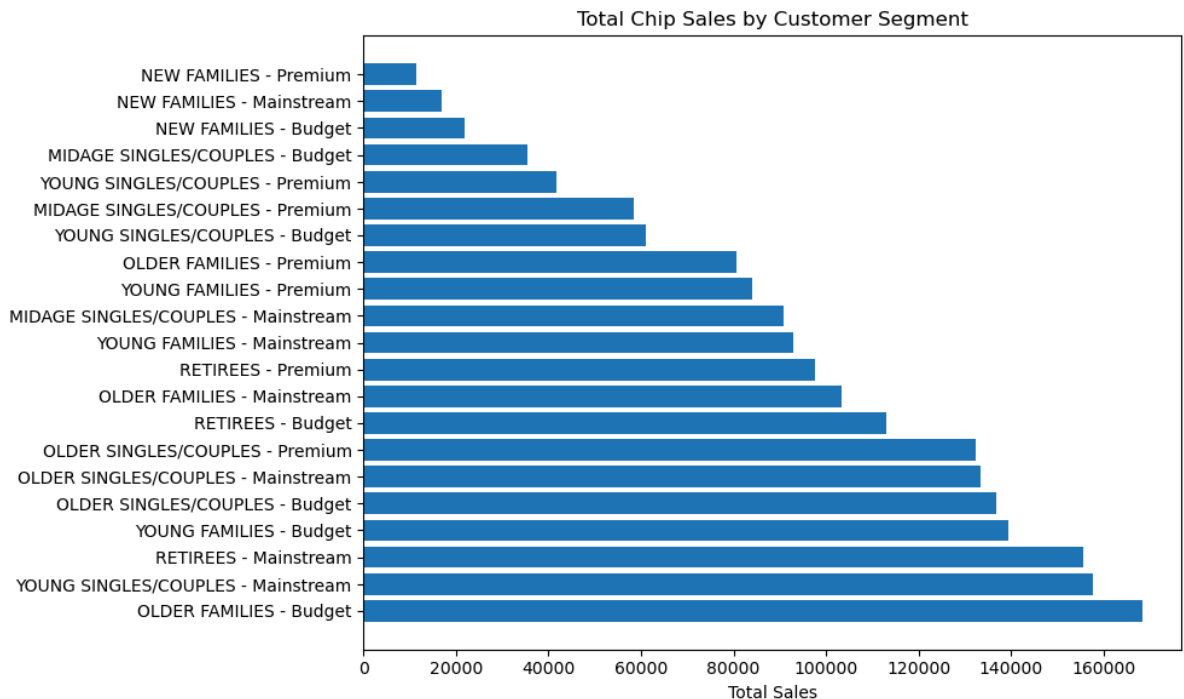
```

# Visualization

# Total sales by segment

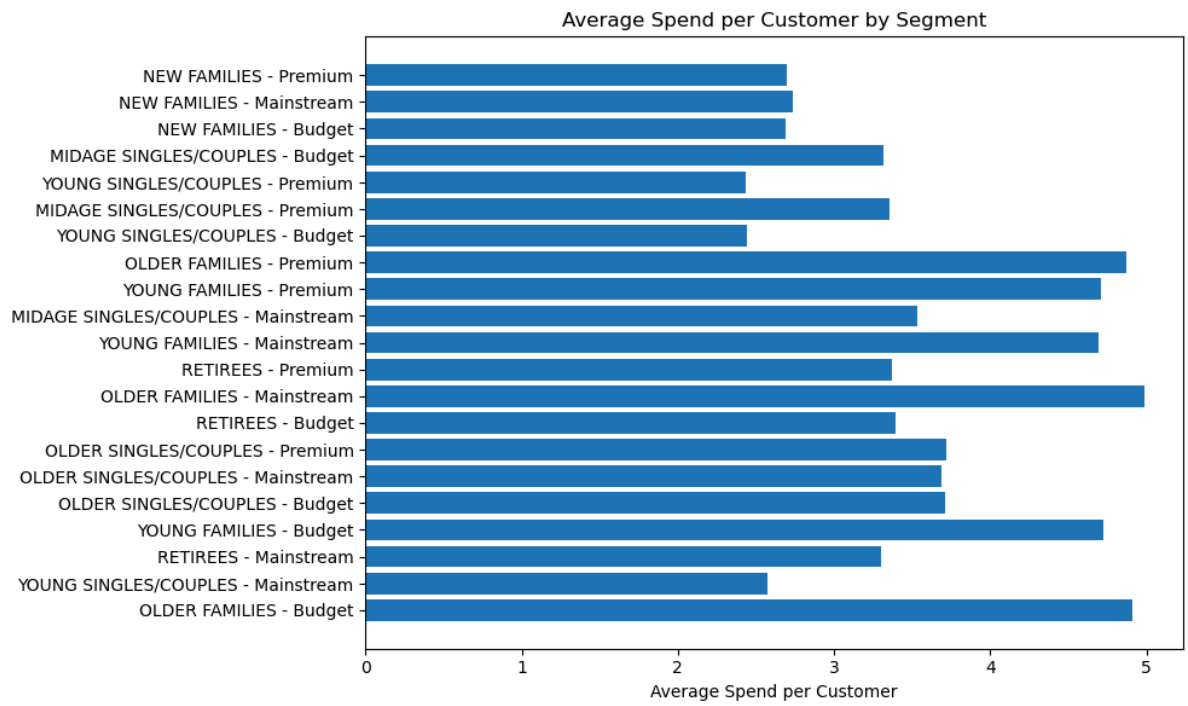
```

```
plt.figure(figsize=(10,6))
plt.barh(segment_summary['segment'], segment_summary['tot_sales'])
plt.xlabel('Total Sales')
plt.title('Total Chip Sales by Customer Segment')
plt.tight_layout()
plt.show()
```



In [134...

```
# Avg spend per customer by segment
plt.figure(figsize=(10,6))
plt.barh(segment_summary['segment'],
segment_summary['Avg_trans_per_cust'])
plt.xlabel('Average Spend per Customer')
plt.title('Average Spend per Customer by Segment')
plt.tight_layout()
plt.show()
```



In []:

